

# Universidad Tecnológica de la Mixteca

## Instituto de Computación



## Algoritmo de Primeros y Siguientes

*Informe de Actividades — Equipo 1*

### INTEGRANTES DEL EQUIPO

- Lujan De la Rosa Cristian Emir
- Cortez Vega Jaime Fabian
- Carrasco Pacheco Levi Josue
- Nicolas Vasquez Joel Osmar
- Vasquez Moreno Zair De Jesus
- Arias Mendoza Angel Adrian

Prof. David Martínez Torres

Compiladores • 502-A • Octubre 2026

*Índice*

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>Análisis</b>                                          | <b>3</b>  |
| <b>Modelo del programa principal</b>                     | <b>4</b>  |
| <b>Asignación de actividades</b>                         | <b>5</b>  |
| <b>Clases y métodos implementados por cada subequipo</b> | <b>7</b>  |
| <b>Avances</b>                                           | <b>9</b>  |
| <b>Conclusiones</b>                                      | <b>11</b> |
| <b>Referencias</b>                                       | <b>12</b> |

## Análisis

Los conjuntos **PRIMERO** y **SIGUIENTE** constituyen estructuras formales fundamentales en el diseño de analizadores sintácticos predictivos (LL(1)). Su cómputo correcto a partir de una gramática libre de contexto determina, respectivamente, qué terminal puede iniciar una derivación (**PRIMERO**) y qué terminal puede aparecer inmediatamente a la derecha de un no terminal en alguna forma sentencial (**SIGUIENTE**).

Para el cálculo de ambos conjuntos se adoptó un enfoque iterativo de *punto fijo*: las reglas de inferencia se aplican de manera repetida hasta que ningún conjunto varía entre dos iteraciones consecutivas. Este método garantiza la convergencia para cualquier gramática libre de contexto finita.

### Algoritmo de Primero

1. Si  $X$  es un símbolo terminal, entonces  $\text{PRIMERO}(X) = \{X\}$ .
2. Si existe la producción  $X \rightarrow \varepsilon$ , se incorpora  $\varepsilon$  a  $\text{PRIMERO}(X)$ .
3. Para cada producción  $X \rightarrow Y_1 Y_2 \cdots Y_n$ :
  - Se incorpora  $\text{PRIMERO}(Y_1) - \{\varepsilon\}$  a  $\text{PRIMERO}(X)$ .
  - Si  $\varepsilon \in \text{PRIMERO}(Y_i)$  para todo  $i < k$ , se incorpora también  $\text{PRIMERO}(Y_k) - \{\varepsilon\}$ .
  - Si  $\varepsilon \in \text{PRIMERO}(Y_i)$  para todo  $i \in \{1, \dots, n\}$ , se incorpora  $\varepsilon$  a  $\text{PRIMERO}(X)$ .

### Algoritmo de Siguiente

1. Se incorpora  $\$$  a  $\text{SIGUIENTE}(S)$ , donde  $S$  denota el símbolo inicial de la gramática.
2. Para cada producción  $A \rightarrow \alpha B \beta$ :
  - Se incorpora  $\text{PRIMERO}(\beta) - \{\varepsilon\}$  a  $\text{SIGUIENTE}(B)$ .
  - Si  $\varepsilon \in \text{PRIMERO}(\beta)$  (o bien  $\beta = \varepsilon$ ), se incorpora  $\text{SIGUIENTE}(A)$  a  $\text{SIGUIENTE}(B)$ .

El sistema implementa ambos algoritmos en Python mediante una arquitectura modular organizada en tres capas: *modelos*, *persistencia* y *presentación*. La capa de modelos define la clase `Gramatica`; la capa de persistencia agrupa los módulos de lectura de archivos y de cómputo de los conjuntos; y la capa de presentación contiene la interfaz gráfica construida con Tkinter.

## Modelo del programa principal

El sistema se estructura en tres capas bien diferenciadas que separan las responsabilidades de cada módulo y facilitan el mantenimiento y la extensibilidad del proyecto.

- **Capa de modelos.** Contiene la clase **Gramatica**, que representa de forma abstracta una gramática libre de contexto: almacena los conjuntos de terminales y no terminales, el símbolo inicial y el diccionario de producciones.
- **Capa de persistencia.** Agrupa tres módulos independientes: `lector_archivo.py`, encargado de parsear el archivo `.txt` y construir el objeto **Gramatica**; `algoritmo_primeros.py`, que calcula los conjuntos PRIMERO mediante iteración de punto fijo; y `algoritmo_siguientes.py`, que calcula los conjuntos SIGUIENTE con la misma estrategia.
- **Capa de presentación.** Contiene los módulos de interfaz gráfica construidos con Tkinter: `estilos.py` define la paleta visual; `tabla_base.py` y `tabla_conjuntos.py` implementan los widgets de tabla reutilizables; `ventana_primeros_siguientes.py` presenta los resultados en pestañas; e `interfaz.py` actúa como punto de entrada de la aplicación.

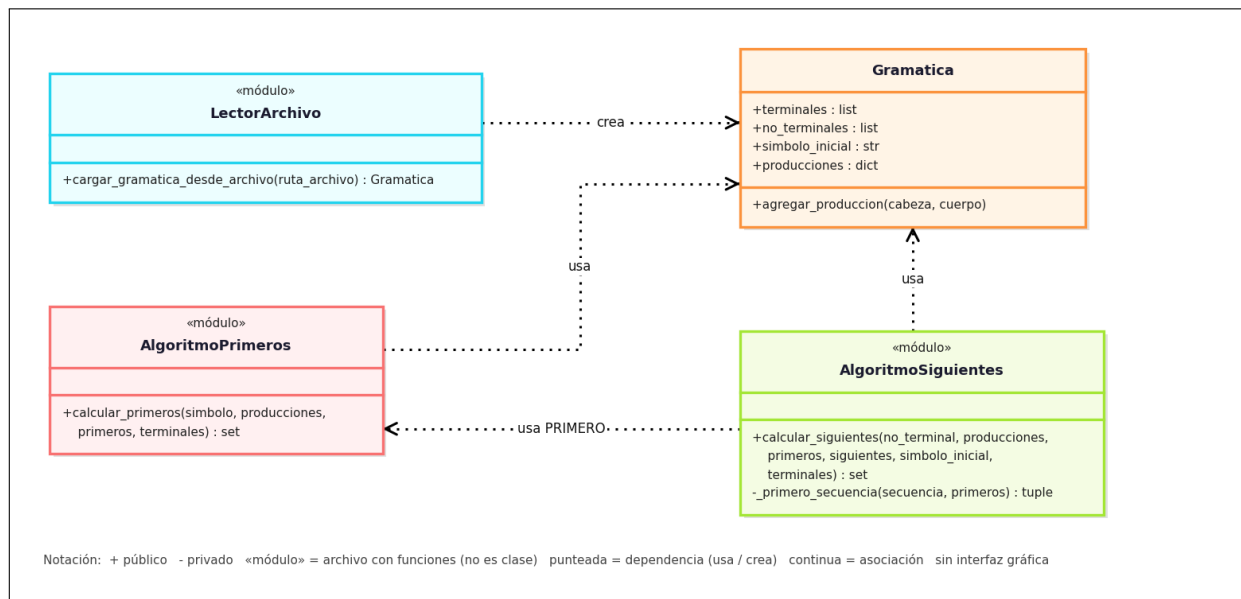


Figura 1: Diagrama de clases.

*Asignación de actividades*

Sub equipos:

1. Cortez Vega Jaime Fabian y Lujan De la Rosa Cristian Emir
2. Arias Mendoza Angel Adrian y Vasquez Moreno Zair De Jesus
3. Carrasco Pacheco Levi Josue y Nicolas Vasquez Joel Osmar

| Equipo | Actividad                                                                                                                                             | Entregas clave                                                                                                                      | Fecha de Entrega |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|------------------|
| Todos  | Diseño de la estructura general del proyecto, definición de la clase <code>Gramatica</code> y comprensión del algoritmo de punto fijo.                |                                                                                                                                     | Semana 1.        |
| 1      | Implementación del algoritmo de PRIMERO y del algoritmo de SIGUIENTE mediante iteración de punto fijo.                                                | Módulos <code>algoritmo_primeros.py</code> y <code>algoritmo_siguientes</code> funcionales con pruebas sobre gramáticas de ejemplo. | Semana 2.        |
| 2      | Implementación de la clase <code>Gramatica</code> y del lector de archivos <code>.txt</code> en sus dos formatos (simple y con encabezados).          | Módulos <code>gramatica.py</code> y <code>lector_archivo.py</code> con manejo de errores y soporte para $\varepsilon$ .             | Semana 2.        |
| 3      | Diseño e implementación de la interfaz gráfica: ventana principal, editor de gramática con numeración de líneas y ventana de resultados con pestañas. | Módulos de presentación integrados en <code>interfaz.py</code> ; interfaz funcional y probada en Windows y macOS.                   | Semana 2.        |

- Es importante que cada integrante del equipo registre su avance diario en el informe de actividades, describiendo las tareas realizadas o los errores encontrados.
- La primera reunión de control de avances tendrá lugar al finalizar la semana 1, donde los sub equipos presentarán su diseño y resolverán dudas.
- La segunda reunión se realizará al cierre de la semana 2 para integrar los módulos de algoritmos y modelo de datos con la interfaz en desarrollo.
- Los días restantes se dedicarán a pruebas de integración, corrección de errores y presentación final con el profesor.

*Clases y métodos implementados por cada subequipo***Cortez Vega y Lujan De la Rosa:**

1. **Módulo Algoritmo PRIMERO** (`algoritmo_primeros.py`): Implementa la función `calcular_primeros`, que calcula los conjuntos PRIMERO para todos los símbolos de la gramática mediante un algoritmo iterativo de punto fijo. La función recibe el símbolo de consulta, el diccionario de producciones, un diccionario mutable de conjuntos `primeros` y la lista de terminales. Inicializa cada terminal con su propio conjunto y cada no terminal con el conjunto vacío; luego itera hasta que no haya cambios.
2. **Módulo Algoritmo SIGUIENTE** (`algoritmo_siguientes.py`): Implementa dos funciones:
  - a. **`_primero_secuencia`**: función auxiliar que calcula PRIMERO de una secuencia de símbolos e indica si toda la secuencia puede derivar en  $\varepsilon$ .
  - b. **`calcular_siguientes`**: función principal que aplica el algoritmo de SIGUIENTE mediante iteración de punto fijo. Inicializa el símbolo inicial con  $\{\$ \}$  y, para cada producción  $A \rightarrow \alpha B \beta$ , propaga `PRIMERO( $\beta$ )` y, cuando corresponde, `SIGUIENTE( $A$ )` hacia `SIGUIENTE( $B$ )`.

**Arias Mendoza y Vasquez Moreno:**

1. **Módulo Gramática** (`gramatica.py`): Define la clase `Gramatica`, que representa una gramática libre de contexto. Sus atributos son: `terminales`, `no_terminales`, `simbolo_inicial` y `producciones` (diccionario de cabeza a lista de cuerpos). El método `agregar_produccion` inserta una nueva regla de producción al diccionario.
2. **Módulo Lector de Archivo** (`lector_archivo.py`): Implementa la función `cargar_gramatica_desde_archivo`, que lee un archivo `.txt` codificado en UTF-8 y construye un objeto `Gramatica`. Soporta dos formatos:
  - a. **Formato simple**: solo producciones; terminales y no terminales se infieren automáticamente cualquier símbolo que encabece una producción es no terminal, el resto son terminales.
  - b. **Formato con encabezados**: las dos primeras líneas declaran explícitamente los terminales y no terminales; las producciones inician en la tercera línea.

Acepta producciones vacías escritas como  $\varepsilon$ , `e` o `epsilon` (sin distinción de mayúsculas) y lanza `ValueError` ante cualquier producción malformada.

**Carrasco Pacheco y Nicolas Vasquez:**

1. **Módulo de Estilos** (`estilos.py`): Define la paleta de colores (**COLORES**), la función `configurar_estilos` que aplica los temas `ttk` globalmente, y la función `crear_superficie` que genera marcos con un diseño visual uniforme para todas las secciones de la interfaz.
2. **Módulo Tabla Base** (`tabla_base.py`): Widget `Treeview` reutilizable que incluye barras de desplazamiento vertical y horizontal. Permite configurar las columnas, sus encabezados, anchos y alineación, y es utilizado como base por los demás módulos de presentación.
3. **Módulo Tabla Conjuntos** (`tabla_conjuntos.py`): Especializa la tabla base para mostrar los conjuntos PRIMERO o SIGUIENTE, presentando un no terminal por fila y su conjunto correspondiente como cadena de símbolos separados por comas.
4. **Módulo Ventana de Resultados** (`ventana_primeros_siguientes.py`): Implementa la clase `VentanaPrimerosSiguientes`, una ventana secundaria con dos pestañas: una para los conjuntos PRIMERO y otra para los conjuntos SIGUIENTE. Cada pestaña muestra una `TablaConjuntos` con los resultados del análisis.
5. **Módulo Interfaz Principal** (`interfaz.py`): Implementa la clase `InterfazCompilador`, punto de entrada de la aplicación. Proporciona:
  - Carga de archivos `.txt` mediante diálogo del sistema operativo.
  - Editor de solo lectura con numeración de líneas para inspeccionar la gramática.
  - Botón **Analizar** que invoca los módulos de PRIMERO y SIGUIENTE y abre la ventana de resultados.
  - Botón **Limpiar** para reiniciar el estado de la aplicación.
  - Barra de estado con mensajes informativos sobre el progreso del análisis.

## Avances

| Fecha      | Lujan De la Rosa Cristian Emir                                                                                       | Cortez Vega Jaime Fabian                                                                                          | Arias Mendoza Angel Adrian                                                                         |
|------------|----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| 30/09/2026 | Conformación de subequipos, análisis de los requerimientos de la Asignación 3 y estructuración general del proyecto. | Análisis de los requerimientos de la Asignación 3 y estructuración general del proyecto.                          | Análisis de los requerimientos de la Asignación 3 y estructuración general del proyecto.           |
| 01/10/2026 | Diseño e implementación de la clase y lógica para la lectura y estructuración de la gramática desde archivo.         | Apoyo en el diseño del módulo de lectura de gramáticas y separación de terminales y no terminales.                | Investigación y planteamiento de las reglas algorítmicas para el cálculo del conjunto de Primeros. |
| 02/10/2026 | Desarrollo de la función base para el reconocimiento del símbolo inicial y producciones de la gramática.             | Creación de estructuras de datos (diccionarios y listas) para almacenar las reglas de producción de forma limpia. | Codificación de la regla 1 y 2 de Primeros (manejo de terminales y épsilon).                       |
| 03/10/2026 | Integración del algoritmo recursivo para evaluar cadenas de símbolos en el cálculo de Primeros.                      | Pruebas unitarias de lectura con la Gramática 1 (expresiones aritméticas).                                        | Implementación de la regla recursiva de Primeros para múltiples símbolos consecutivos.             |
| 04/10/2026 | Depuración de errores lógicos en el manejo de producciones vacías durante el cálculo de Primeros.                    | Revisión y depuración del flujo de datos entre el lector de archivos y los algoritmos sintácticos.                | Pruebas de validación del módulo de Primeros utilizando las Gramáticas 2 y 3.                      |
| 05/10/2026 | Pruebas con la Gramática 4 y verificación de casos especiales con épsilon.                                           | Pruebas con la Gramática 5 y ajuste de formato en la lectura de archivos de texto.                                | Refinamiento de la lógica de Primeros para evitar bucles infinitos en gramáticas recursivas.       |
| 06/10/2026 | Revisión general de la funcionalidad del programa con la Gramática 6 y pruebas finales.                              | Verificación de salidas en pantalla comparadas con los ejemplos teóricos del profesor.                            | Pruebas finales y depuración de detalles menores en los conjuntos calculados.                      |
| 07/10/2026 | Redacción y consolidación final del informe de actividades y documentación del proyecto.                             | Apoyo en la revisión y pulido del informe de actividades y evidencias de la práctica.                             | Integración de las conclusiones y revisión de la rúbrica de evaluación.                            |

| Fecha      | Vasquez Moreno Zair De Jesus                                                                                                                          | Nicolas Vasquez Joel Osmar                                                                                                                | Carrasco Pacheco Levi Josue                                                                                                  |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| 30/09/2026 | Análisis de los requerimientos de la Asignación 3 y estructuración general del proyecto.                                                              | Análisis de los requerimientos de la Asignación 3 y estructuración general del proyecto.                                                  | Análisis de los requerimientos de la Asignación 3 y estructuración general del proyecto.                                     |
| 01/10/2026 | Investigación y práctica manual con el algoritmo de Primeros sobre gramáticas sencillas para comprender su funcionamiento previo a la implementación. | Prácticas manuales con el algoritmo de Siguientes sobre gramáticas dadas por el profesor para entender su funcionamiento.                 | Investigación y desarrollo manual de prácticas con el algoritmo de Siguientes para entender su funcionamiento.               |
| 02/10/2026 | Apoyo en la implementación de la clase Gramatica y revisión de la estructura de producciones generada por el lector de archivos.                      | Apoyo en el modelo de gramática en Python elaborado para poder usarlo en los dos algoritmos de conjuntos, para una estructuración limpia. | Diseño e implementación de la función que recorre las producciones para identificar el contexto de cada no terminal.         |
| 03/10/2026 | Elaboración de pruebas manuales del algoritmo de Siguientes para validar su comportamiento con la Gramática 1.                                        | Elaboración de la función que implementa las reglas de funcionamiento del algoritmo de Siguientes.                                        | Elaboración de la función que calcula los primeros de $\beta$ para poder añadirlos a los Siguientes dada la regla utilizada. |
| 04/10/2026 | Apoyo en la revisión del módulo de Primeros y verificación de resultados con las Gramáticas 2 y 3.                                                    | Pruebas de integración del módulo de Siguientes con el lector de archivos y la clase Gramatica.                                           | Pruebas de la función que calcula los primeros de $\beta$ con gramáticas que contienen épsilon.                              |
| 05/10/2026 | Pruebas del algoritmo de Siguientes con las Gramáticas 4 y 5 para verificar su corrección.                                                            | Depuración de la función que calcula los primeros de $\beta$ para casos con múltiples producciones.                                       | Verificación del módulo de Siguientes con la Gramática 5 y corrección de detalles en la propagación de conjuntos.            |
| 06/10/2026 | Verificación de los conjuntos de Siguientes calculados comparados con los resultados esperados del profesor.                                          | Pruebas finales del algoritmo de Siguientes con todas las gramáticas de prueba disponibles.                                               | Revisión general del módulo de Siguientes y validación de resultados con los ejemplos teóricos.                              |
| 07/10/2026 | Apoyo en la redacción del reporte de actividades y revisión de la documentación del proyecto.                                                         | Redacción de los elementos faltantes dentro del reporte de actividades, integración del diagrama UML.                                     | Apoyo en la redacción final del documento, revisión general de todos los puntos y corrección de faltas ortográficas.         |

## Conclusiones

El desarrollo del presente proyecto permitió al equipo comprender de manera rigurosa y práctica el papel que desempeñan los conjuntos **PRIMERO** y **SIGUIENTE** en la construcción de analizadores sintácticos predictivos. A diferencia del análisis léxico que reconoce unidades léxicas mediante autómatas finitos deterministas el análisis sintáctico predictivo exige razonar sobre las derivaciones posibles de la gramática; los conjuntos **PRIMERO** y **SIGUIENTE** constituyen la herramienta formal que hace posible tomar tales decisiones de manera determinista y sin necesidad de retroceso.

La implementación del algoritmo iterativo de punto fijo exigió un análisis cuidadoso de las dependencias entre los conjuntos de distintos no terminales: una modificación en  $\text{PRIMERO}(X)$  puede propagarse hacia los conjuntos **SIGUIENTE** de otros símbolos en cadena. El manejo correcto de dichas dependencias mediante la repetición controlada de las reglas de inferencia representó el núcleo algorítmico del proyecto y reforzó la comprensión teórica de la convergencia de procesos iterativos de punto fijo.

Desde la perspectiva del diseño de software, la separación en tres capas modelos, persistencia y presentación facilitó la distribución del trabajo entre los sub equipos y permitió desarrollar y verificar cada módulo de forma independiente antes de la fase de integración. La interfaz gráfica construida con Tkinter añadió valor práctico al sistema al hacer accesible el cálculo de los conjuntos a cualquier usuario, sin requerir conocimiento del código fuente.

En síntesis, el proyecto consolidó la comprensión de la segunda fase en el diseño de compiladores y reforzó buenas prácticas de ingeniería de software modularidad, pruebas incrementales y documentación que serán aplicadas en las etapas subsecuentes del curso.

## Referencias

Martínez Torres, D. (s. f.). *Tema 3. Análisis sintáctico* [Diapositivas de PowerPoint]. Instituto de Computación, Universidad Tecnológica de la Mixteca.

Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2008). *Compiladores: principios, técnicas y herramientas* (2.1 ed.). Pearson Educación.

Mogensen, T. Æ. (2017). *Introduction to Compiler Design* (2nd ed.). Springer.